Sign in

QwenLM / Qwen3-Coder Public

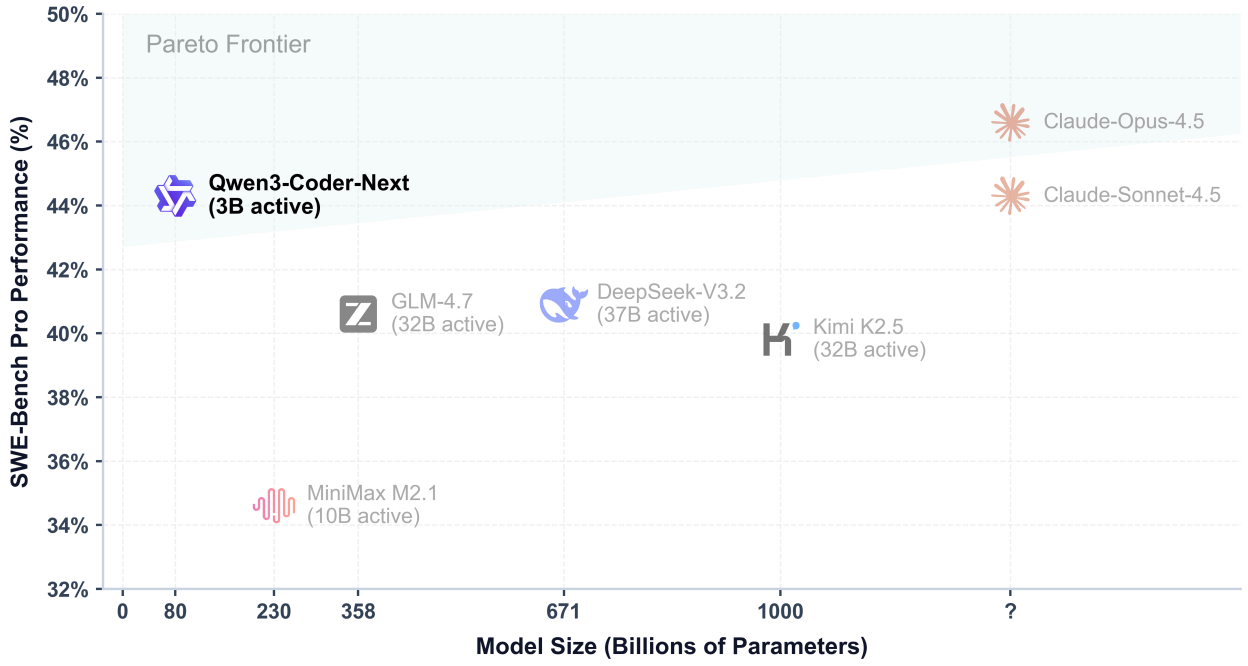
Code Issues 100 Pull requests 15 Actions Projects Security and

main Go to file Code

cyente Merge pull request #575 from SHUMKASHUN/patch-1 33bc6aa · 2 weeks ago

assets	qwen3-coder-next released	2 months ago
demo	Add generation parameters (temperat...	2 years ago
examples	Revise fim examples	last year
finetuning	fix(finetuning): supported transformer...	8 months ago
qwencoder-eval	update data.zip	4 months ago
.gitignore	add: livecode-bench for QwQ-32B-Pr...	last year
README.md	Update bibtex	last month
qwen3_coder_next_tech_repor...	Update tech report	2 months ago
requirements.txt	Update requirements.txt	9 months ago

README



💜 Qwen Chat | 🤗 Hugging Face | 🤖 ModelScope | 📑 Blog | 📖 Documentation

🌍 WebDev | 💬 WeChat (微信) | 🫨 Discord | 📄 Arxiv | 👽 Qwen Code

Visit our Hugging Face or ModelScope organization (click links above), search checkpoints with names starting with `Qwen3-Coder-`, and you will find all you need! Enjoy!

Table of Contents

- Introduction
  - Key Features
- Basic Information
- Quick Start
  - 👉 Chat with Qwen3-Coder
    - Fill in the middle with Qwen3-Coder
- Use Cases
  - Example: Releasing a Website
  - Example: Desktop Tidy
  - Example: Zombies vs. Plants
  - Example: Sound ASCII Art
  - Example: Vibe Checking
  - Example: Parkour Game
- Star History
- Citation

- Contact Us

Qwen3-Coder-Next: Pushing Small Hybrid Models on Agentic Coding

Introduction

We are announcing Qwen3-Coder, our most agentic code model to date. **Qwen3-Coder** is available in multiple sizes, **Qwen3-Coder-480B-A35B-Instruct**, **Qwen3-Coder-30B-A3B-Instruct**, **Qwen3-Coder-Next**, offering exceptional performance in both coding and agentic tasks.

Qwen3-Coder-Next, an open-weight language model designed specifically for coding agents and local development. Built on top of **Qwen3-Next-80B-A3B-Base**, which adopts a novel architecture with hybrid attention and MoE, Qwen3-Coder-Next has been agentically trained at scale on large-scale executable task synthesis, environment interaction, and reinforcement learning, obtaining strong coding and agentic capabilities with significantly lower inference costs.

Key Features

💻 **Efficiency-Performance Tradeoff**: among open models on **Agentic Coding**, **Agentic Browser-Use**, and other foundational coding tasks, achieving results comparable to Claude Sonnet.

🛠️ **Scaling Agentic Coding**: supporting most platforms such as **Qwen Code**, **CLINE**, **Claude Code**, featuring a specially designed function call format;

📚 **Long-context Capabilities**: with native support for **256K** tokens, extendable up to **1M** tokens using Yarn, optimized for repository-scale understanding.

Basic Information

1. ✨ Supporting long context understanding and generation with the context length of 256K tokens;
2. ✨ Supporting 358 coding languages;

▶ Click to view all supported languages

3. ✨ Retain strengths in math and general capabilities from base model.

> Important
>
> Qwen3-Coder function calling relies on our new tool parser in both **SGLang** and **vLLM** here.
>
> We updated both the special tokens and their corresponding token ids, in order to maintain consistency with Qwen3. Please make sure to use the new tokenizer.

model name	type	length	Download
Qwen3-Coder-Next	instruct	256k	🤗 Hugging Face • 🤖 ModelScope
Qwen3-Coder-Next-Base	base	256k	🤗 Hugging Face • 🤖 ModelScope
Qwen3-Coder-480B-A35B-Instruct	instruct	256k	🤗 Hugging Face • 🤖 ModelScope
Qwen3-Coder-30B-A3B-Instruct	instruct	256k	🤗 Hugging Face • 🤖 ModelScope
Qwen3-Coder-Next-FP8	instruct	256k	🤗 Hugging Face • 🤖 ModelScope
Qwen3-Coder-Next-GGUF	instruct	256k	🤗 Hugging Face • 🤖 ModelScope
Qwen3-Coder-480B-A35B-Instruct-FP8	instruct	256k	🤗 Hugging Face • 🤖 ModelScope
Qwen3-Coder-30B-A3B-Instruct-FP8	instruct	256k	🤗 Hugging Face • 🤖 ModelScope

Detailed performance and introduction are shown in this 📑 blog.

Quick Start

> Important
>
> **Qwen3-Coder** are instruct models for chatting;
>
> This model supports only non-thinking mode and does not generate `<think></think>` blocks in its output. Meanwhile, specifying `enable_thinking=False` is no longer required.

👉 Chat with Qwen3-Coder

You can write several lines of code with `transformers` to chat with Qwen3-Coder-Next. Essentially, we build the tokenizer and the model with the `from_pretrained` method, and we use the generate method to perform chatting with the help of the chat template provided by the tokenizer. Below is an example of how to chat with **Qwen3-Coder-Next**:

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model_name = "Qwen/Qwen3-Coder-Next"

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",
    device_map="auto"
)
tokenizer = AutoTokenizer.from_pretrained(model_name)

prompt = "write a quick sort algorithm."
messages = [
    {"role": "user", "content": prompt}
]
text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True
)
model_inputs = tokenizer([text], return_tensors="pt").to(model.device

generated_ids = model.generate(
    **model_inputs,
    max_new_tokens=65536
)
generated_ids = [
    output_ids[len(input_ids):] for input_ids, output_ids in zip(mode
]

response = tokenizer.batch_decode(generated_ids, skip_special_tokens=
```

The `apply_chat_template()` function is used to convert the messages into a format that the model can understand. The `add_generation_prompt` argument is used to add a generation prompt, which refers to `<|im_start|>assistant\n` to the input. Notably, we apply the ChatML template for chat models following our previous practice. The `max_new_tokens` argument is used to set the maximum length of the response. The `tokenizer.batch_decode()` function is used to decode the response. In terms of the input, the above messages are an example to show how to format your

dialog history and system prompt. You can use the other sizes of instruct models in the same way.

Fill in the middle with Qwen3-Coder

The code insertion task, also referred to as the "fill-in-the-middle" challenge, requires the insertion of code segments in a manner that bridges the gaps within a given code context. For an approach aligned with best practices, we recommend adhering to the formatting guidelines outlined in the paper "Efficient Training of Language Models to Fill in the Middle" [arxiv].

> **Important**
>
> It should be noted that FIM is supported in every version of Qwen3-Coder. Qwen3-Coder-Next is shown here as an example.

The prompt should be structured as follows:

```
prompt = '<|fim_prefix|>' + prefix_code + '<|fim_suffix|>' + suffix_c
```

Following the approach mentioned, an example would be structured in this manner:

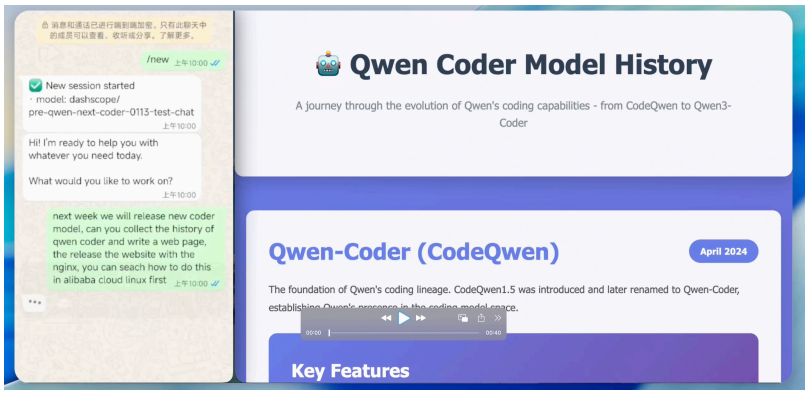
```
from transformers import AutoTokenizer, AutoModelForCausalLM
# load model
device = "cuda" # the device to load the model onto

TOKENIZER = AutoTokenizer.from_pretrained("Qwen/Qwen3-Coder-Next")
MODEL = AutoModelForCausalLM.from_pretrained("Qwen/Qwen3-Coder-Next",


input_text = """<|fim_prefix|>def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    <|fim_suffix|>
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quicksort(left) + middle + quicksort(right)<|fim_middle|>"

messages = [
    {"role": "system", "content": "You are a code completion assistan
    {"role": "user", "content": input_text}
]


text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
```

```
    add_generation_prompt=True
)
model_inputs = TOKENIZER([text], return_tensors="pt").to(model.device

# Use `max_new_tokens` to control the maximum output length.
eos_token_ids = [151659, 151661, 151662, 151663, 151664, 151643, 1516
generated_ids = MODEL.generate(model_inputs.input_ids, max_new_tokens
# The generated_ids include prompt_ids, we only need to decode the to
output_text = TOKENIZER.decode(generated_ids[len(model_inputs.input_i

print(f"Prompt: {input_text}\n\nGenerated text: {output_text}")
```

Use Cases

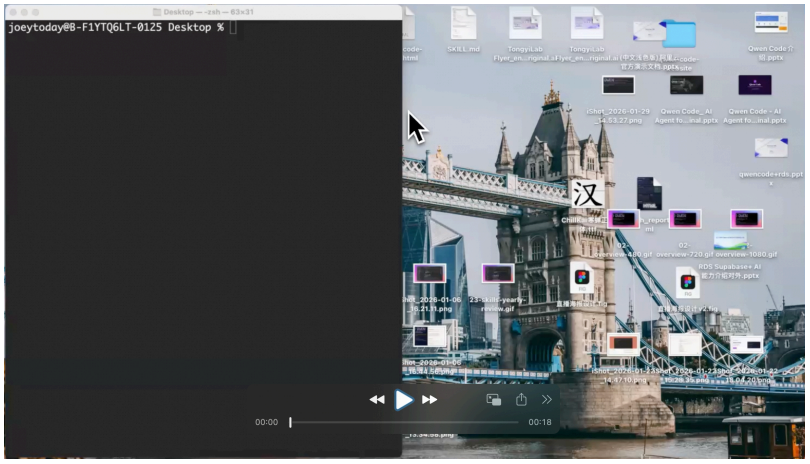
Example: Releasing a Website

▶ Prompt with OpenClaw



Example: Desktop Tidy

▶ Prompt with Qwen Code

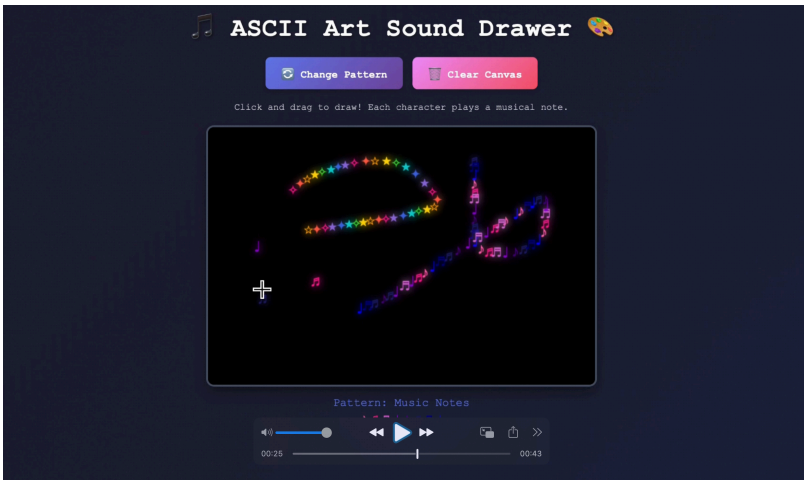


Example: Zombies vs. Plants

▶ Prompt with Claude Code

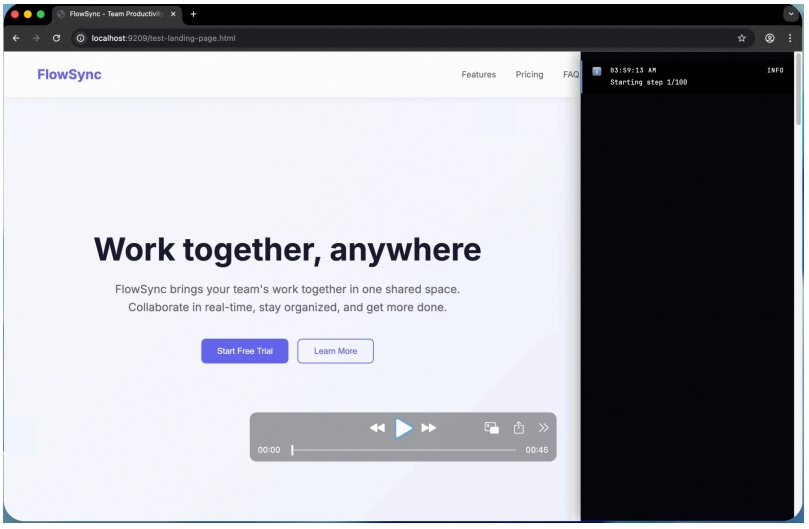
Example: Sound ASCII Art

▶ Prompt with Cline



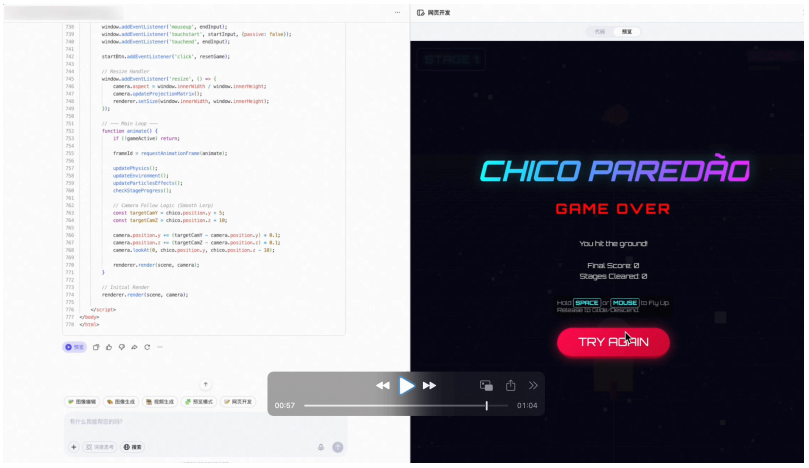
Example: Vibe Checking

▶ Prompt with Browser Use Agent

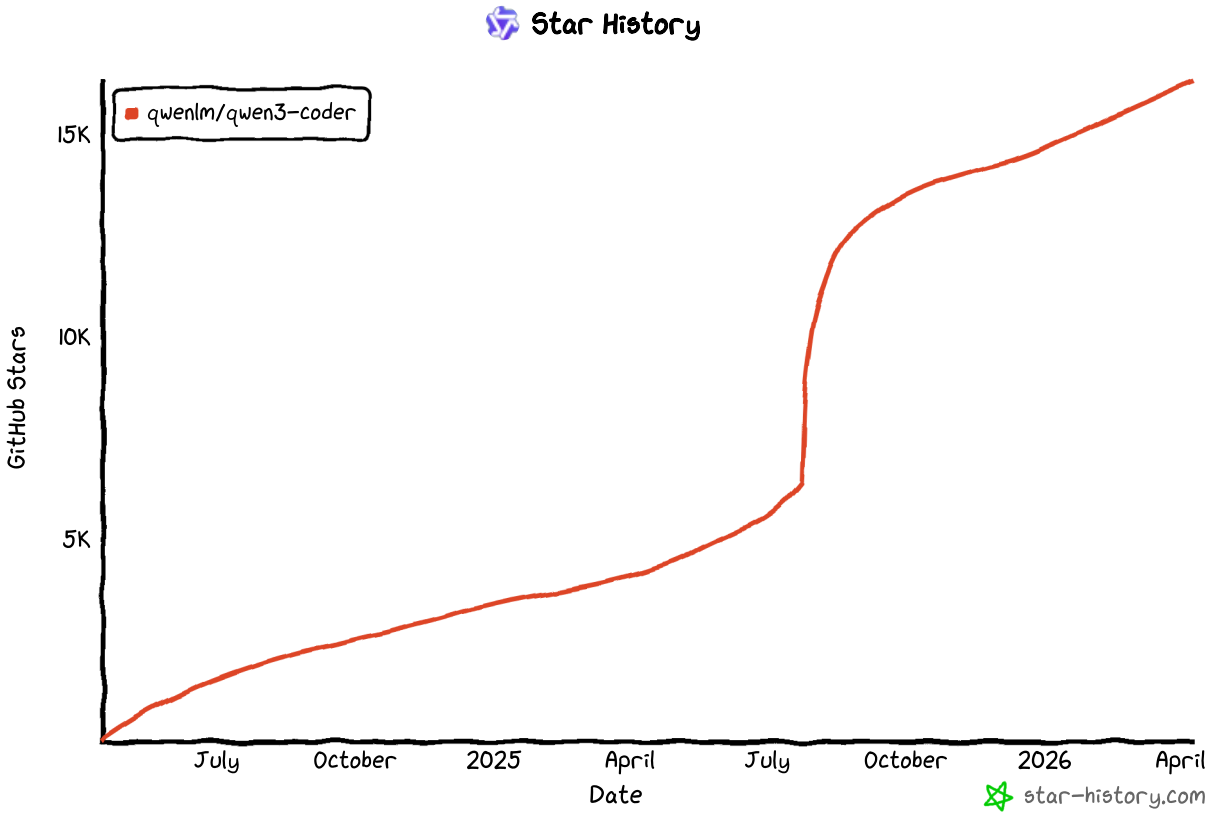


Example: Parkour Game

▶ Prompt with Qwen Chat Web Dev



Star History



Citation

If you find our work helpful, feel free to give us a cite.

```
@article{Qwen3-Coder-Next,
    title={Qwen3-Coder-Next Technical Report},
    author={Ruisheng Cao and Mouxiang Chen and Jiawei Chen and Zeyu C
```

```
    journal={arXiv preprint arXiv:2603.00729},
    year={2026},
}
```

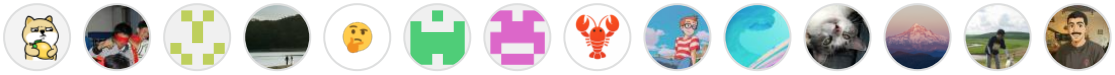
Releases

No releases published

Packages

No packages published

Contributors 34



+ 20 contributors

Languages

Python 85.2% **Go** 12.2% **Shell** 1.8% **Tree-sitter Query** 0.3% **JavaScript** 0.2%
Makefile 0.1% **Other** 0.2%